

Chapter Four

1. Error detection and correction codes

The idea of error detection and/or correction is to add extra bits to the digital message that enable the receiver to detect or correct errors with limited capabilities. These extra bits are called parity bits. If we have k bits, r parity bits are added, then the transmitted digits are:

$$n = r + k$$

Here n called code word denoted as (n, k) . The efficiency or code rate is equal to k/n .

Two basic approaches to error correction are available, which are:

a- Automatic-repeat-request (ARQ): Discard those frames in which errors are detected. - For frames in which no error was detected, the receiver returns a positive acknowledgment to the sender. - For the frame in which errors have been detected, the receiver returns negative acknowledgement to the sender.

b- Forward error correction (FEC): Ideally, FEC codes can be used to generate encoding symbols that are transmitted in packets in such a way that each received packet is fully useful to a receiver to reassemble the object regardless of previous packet reception patterns. The most applications of FEC are: Compact Disc (CD) applications, digital audio and video, Global System Mobile (GSM) and Mobile communications.

2. Basic definitions :

- Systematic and nonsystematic codes: If $(a$'s) information bits are unchanged in their values and positions at the transmitted code word, then this code is said to be systematic (also called block code) where: Input data $D = [a_1, a_2, a_3, \dots, a_k]$, The output systematic code word (n, k) is:
 $C = [a_1, a_2, a_3, \dots, a_k, c_1, c_2, c_3, \dots, c_r]$

However if the data bits are spread or changed at the output code word then, the code is said to be nonsystematic. The output of nonsystematic code word of (n, k) :
 $C = [c_2, a_1, c_1, a_3, a_2, c_3, \dots]$

- Hamming Distance (HD): It is important parameter to measure the ability of error detection. It the number of bits that differ between any two codewords C_i and C_j denoted by d_{ij} . For a binary (n, k) code with 2^k possible codewords, then minimum HD (d_{min}) is $\min(d_{ij})$,

where: $n \geq d_{ij} \geq 0$

For any code word, the possible error detection is:

$$2t = d_{min} - 1$$

For example, if $d_{min} = 4$, then it is possible to detect 3 errors or less. The possible error correcting is:

$$t = \frac{d_{min} - 1}{2}$$

So, for $d_{min} = 4$, it is possible to correct only one bit.

Example 1: Find the minimum HD between the following codewords. Also determine the possible error detection and the number of error correction bits.

$$C1 = [100110011], C2 = [111101100] \text{ and } C3 = [101100101]$$

Solution:

Here $d_{12} = 6$, $d_{13} = 4$ and $d_{23} = 3$, hence $d_{min} = 3$.

The possible error detection $2t = d_{min} - 1 = 2$.

The possible error correction $t = d_{min}-1/2 = 1$.

-Hamming Weight: It is the number of 1's in the non-zero codeword C_i , denoted by w_i . For example the codewords of $C1 = [1011100]$, $C2 = [1011001]$, $w1 = 4$, and $w2 = 3$ respectively. If we have two valid codewords- all ones and all zeros, in this case $HD = w_i$.

3. Parity check codes (Error detection):

It is a linear block code (systematic codes). In this code, an extra bit is added for each k information and hence the code rate (efficiency) is $k/(k+1)$. At the receiver, if the number of 1's is odd, then the error is detected. The minimum Hamming distance for this category is $d_{min} = 2$, which means that the simple parity code is a single-bit error-detecting code; it cannot correct any error. There are two categories in this type: even parity (ensures that a code word has an even number of 1's) and odd parity (ensures that a code word has an odd number of 1's) in the code word.

Example 2: an even parity-check code of (5, 4), which means that $k = 4$ and $n = 5$.

Data word	Code word	Data word	Code word
0010	00101	0110	01100
1010	10100	1000	10001

4. Repetition codes:

The repetition code is one of the most basic error-correcting codes. The idea of the repetition code is to just repeat the message several times. The encoder is a simple device that repeats r times. For example, if we have a (3, 1) repetition code, then encoding the signal $m = 101001$ yields the code $c = 111000111000000111$.

Suppose we received a (3, 1) repetition code and are decoding the signal $c = 110001111$. The decoded message is $m=101$. For $(r, 1)$ repetition code, an error correcting capacity of $r/2$ (i.e., it will correct up to $r/2$ errors in any code word). In other words, the $d_{min}=r$, or increasing the correction capability depending on the r value. Although this code is very simple, it is also inefficient and wasteful because it uses only (2, 1) repetition code, which doubles the bandwidth and thus doubles the cost.

Example 3: A receiver of (5, 1) repetition code has received the following code words: 100100011010101, 100011001100111, 111000000111110:

- Determine the decoded data word and transmitted code words.
- Calculate the minimum Hamming distances and error correction capability.

Solution:

- a- It must sort each received code word into groups, each one contain 5 bits as follow:

$C_1=[10010, 00110, 10101]$, if the 1s is more than 0s, then the decoded bit is =1 and vise versa.

So that $m_1 = [001]$.

Similarly: $C_2=[10001, 10011, 00111]$ then $m_2 = [011]$

Also, $C_3=[11100, 00001, 11110]$ and $m_3 = [101]$

- b- For repetition code the minimum Hamming distance $r=5$. The error correction capability= $\left\lfloor \frac{r-1}{2} \right\rfloor = \frac{4}{2} = 2 \text{ bits}$

Example 4: A receiver of (5, 1) repetition code has been received the following code words:

(100100011010101, 100011001100111, 111000000111110)

- Determine the decoded data word and transmitted code words.
- Calculate the minimum Hamming distances and error correction capability.

Solution:

Splitting each RX code vector into a groupe of “5” bits with the application of “MAJORITY LOGIC PRINCIPLE” for each groupe as shown below:

(10010 00110 10101, 10001 10011 00111, 11100 00001 11110)										
0	0	1		0	1	1		1	0	1

- a) Decoded data word: $m1 = [001]$, $m2 = [011]$ and $m3 = [101]$. Therefore, TX (Transmitted) code- words are:

$C1 = 000000000011111$, $C2 = 000001111111111$, $C3 = 111110000011111$.

- b) Hamming distance: $d1-2=5$, $d1-3=5$, $d2-3=10$, So that $d_{min}=5$

- Error detection (E.D) = $d_{min}-1=5-1=4$ bits.

- Error correction (E.C) = $[(d_{min}-1)/2] = [(5-1)/2] = 2$.

OR Since the repetition is $r=5$, the error correction capability is $[(r-1)/2] = [4/2] = 2$ bits.

5. Convolutional Code

Convolutional coding were introduced in 1955 by Elias. Convolutional codes offer an approach to error control coding substantially different from that of block codes. Convolutional encoders are linear and time-invariant system given by the convolution of a binary data stream with generator sequences. They can be implemented by shift registers. Can achieve a larger coding gain than can be achieved using a block coding with the same complexity. The information, consisting of individual bits, is fed into a shift register for encoding.

A convolutional encoder is generally characterized in form **(n, k, K) or $(k/n, K)$**

Where: k is the input bits (in practice, usually $k=1$), n is the number of output bits. And K is the constraint length of the of the convolutinal code (where the encoder has $K-1$ memory elements).

S is the number of memory elements

$R_c = k/n$ is the coding rate, determining the number of data bits per coded bit.

The encoder shown is a (2, 1, 3), 1/2 code rate, and 4 possible states.

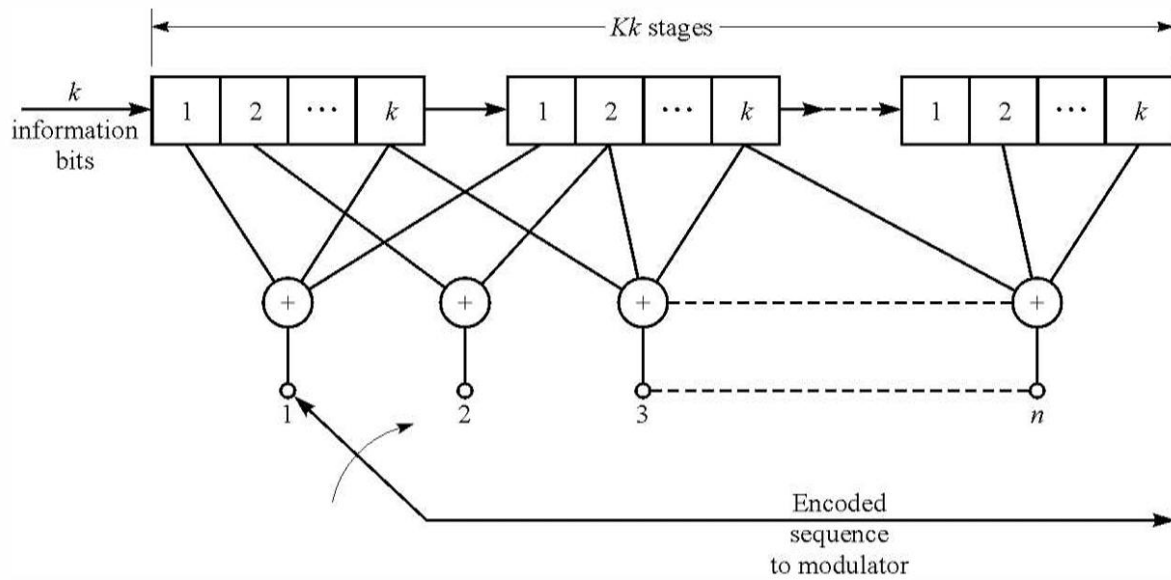


Figure 1: convolutional encoder.

The convolutional encoder (rate $\frac{1}{2}$, $K=3$) shown in Figure 2 consists of 3 shift registers, where the first one takes the incoming data bit, and the rest form the encoder's memory.

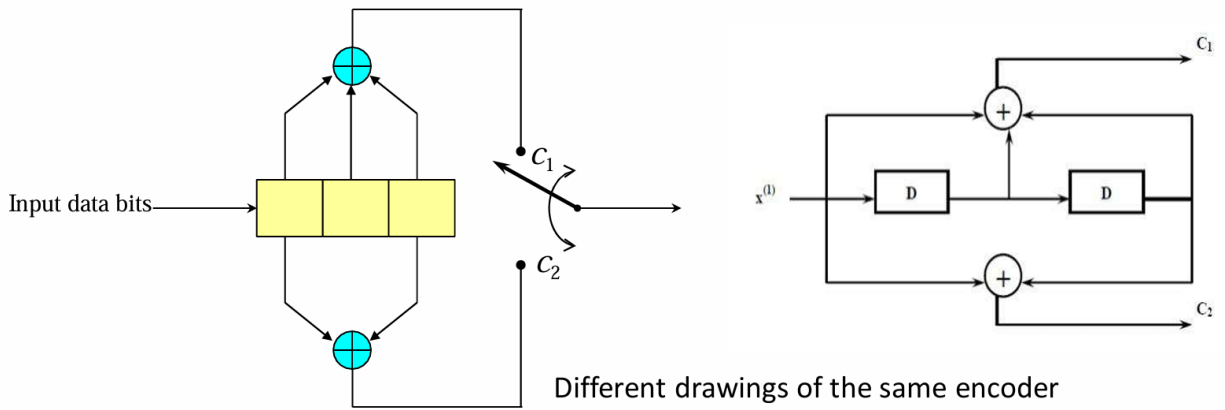


Figure 2: Convolutional encoder (rate $\frac{1}{2}$, $K=3$).

5.1.Non-Systematic Convolutional (NSC) Codes

For example, a Non-Systematic Convolutional (NSC) Code with a constraint length $K = 3$ and a generator polynomial:

$$G(D) = [1 + D, 1 + D + D^2]$$

In this case $1 + D^2$ is 101 and $1 + D + D^2$ is 111.

101 is 5_8 , where the subscript 8 indicates it is an octal number. Similarly, 111 is 7_8 . Therefore, this is the $(7, 5)_8$ NSC code.

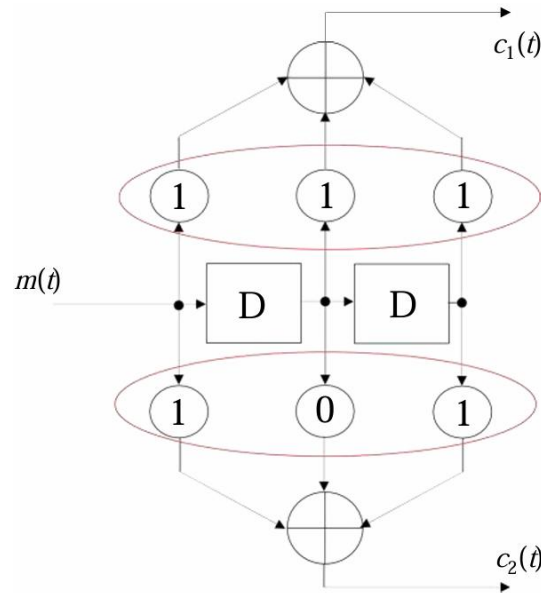


Figure 3: $(7, 5)_8$ NSC encoder.

5.2.State Table of a Convolutional $(7, 5)_8$ NSC Code

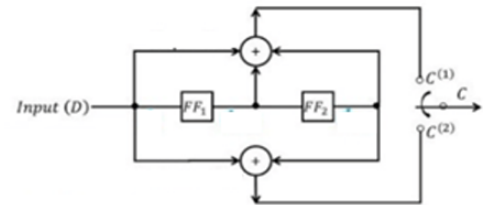
The memory elements can contain four possible values: 00, 01, 10, 11. These values are called states, and only certain state transitions are allowed. For example, we cannot go from state 00 to state 11 in one step, nor can we stay in state 10. All possible state transitions, along with their corresponding input and outputs, can be determined from the encoder and stored in a state table. The state table for the $(7, 5)_8$ NSC code is shown below:

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0

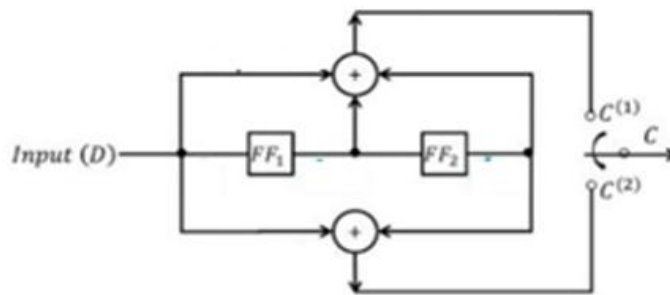
Example 5: Consider the convolutional encoder with $k = 1$, $n = 2$, and $K = 3$ shown below. Sketch a diagram for the encoder and estimate the state table. Consider $g_1 = [111]$ and $g_2 = [101]$.

Solution:

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0



Example 6: Consider the convolutional encoder with $k = 1$, $n = 2$ and $K = 3$ shown below. Estimate the state table to encode the message (1101100)?

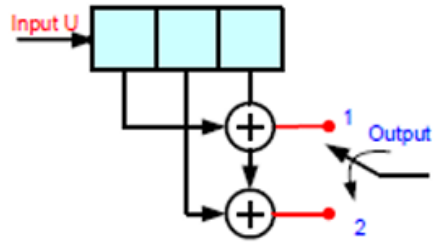


Solution:

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0

Code word = 11 01 01 00 01 01 11

Example 7: An encoder diagram is shown in Figure. Find the encoder output using the state table for an input sequence 1 0 0 1 0 1 0.



Solution:

Input bit	Register Contents	State at time t_i	State at time $t_i + 1$	Output
-	000	00	00	00
1	100	00	10	11
0	010	10	01	01
0	001	01	00	11
1	100	00	10	11
0	010	10	01	01
1	101	01	10	00
0	010	10	01	01

The code word is: 11 01 11 11 01 00 01

5.3.State diagram of a Convolutional Code

A compact form of the state table is the state diagram. The state is represented by the content of the memory, i.e., the $(K-1)k$ previous bits, namely, the $(K-1)k$ bits contained in the first $(K-1)k$ stages of the shift register. Hence, there are $2^{(K-1)k}$ states.

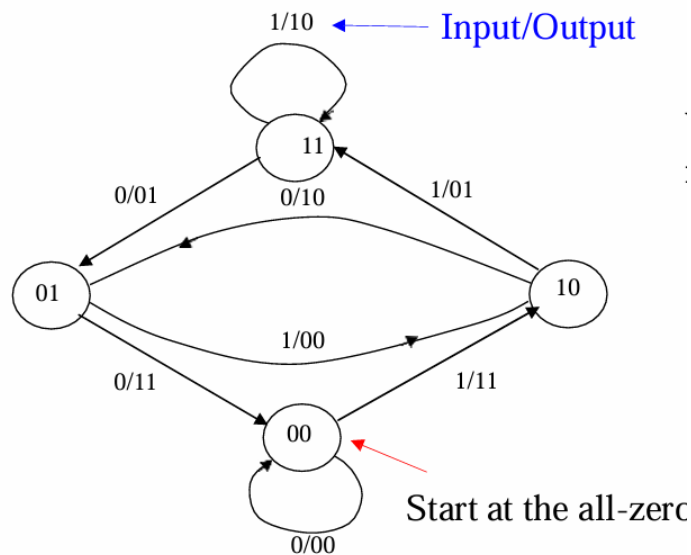
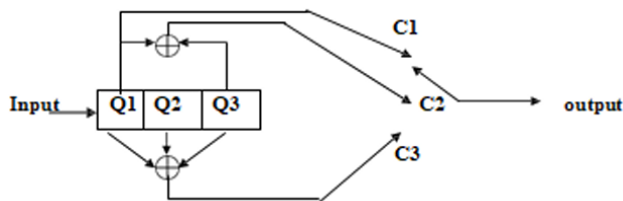


Figure 3: State diagram of a Convolutional Code.

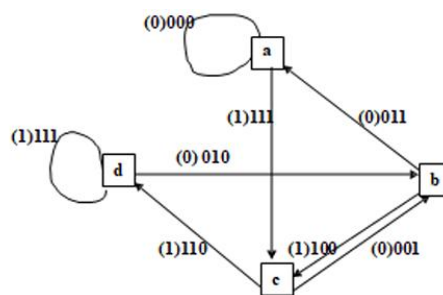
Example 8: Consider the convolutional encoder with $K=3$, $k=1$, $n=3$ shown below. Draw the state diagram of the convolutional encoder.



Solution:

Input	present state of Q2Q3	next state of Q2Q3	output C1C2C3
0	00(a)	00(a)	000
1	00(a)	10(c)	111
0	01(b)	00(a)	011
1	01(b)	10(c)	100
0	10(c)	01(b)	001
1	10(c)	11(d)	110
0	11(d)	01(b)	010
1	11(d)	11(d)	101

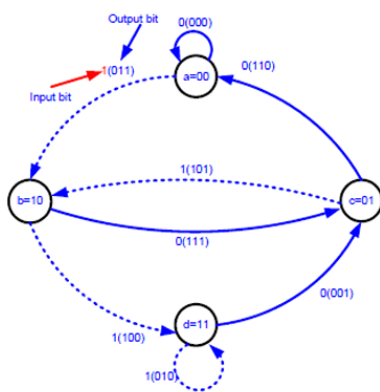
State Table



State diagram

Example 9: Estimate the state table and draw the state diagram and diagram for the $K=3$, rate= $1/3$ encoder. Consider code generated by: $g_1(X) = X+x^2$, $g_2(X) = 1+X+x^2$, $g_3(X) = 1+X$.

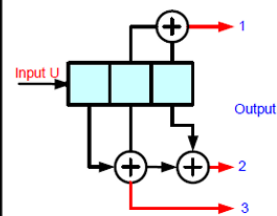
Solution:



State diagram

Input bit	State at time t_i	State at time $t_i + 1$	Output
0	00	00	000
1	00	10	011
0	01	00	110
1	01	10	101
0	10	01	111
1	10	11	100
0	11	01	001
1	11	11	010

State table



Encoder

5.4. Trellis Diagrams of a Convolutional Code

We can represent all possible state transitions with a graph, where nodes represent states and edges represent transitions. Each edge is labelled with an input and its corresponding coded output. By making copies of the graph and joining them together, we create a trellis with each path corresponding to a codeword.

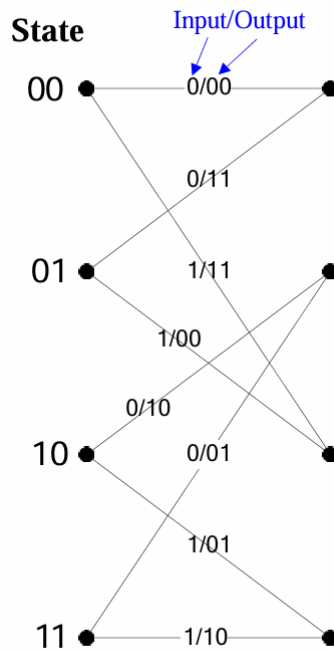
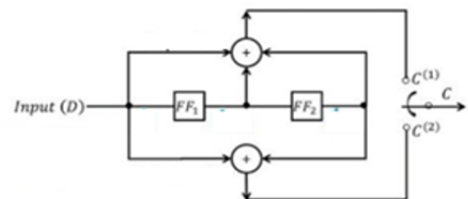


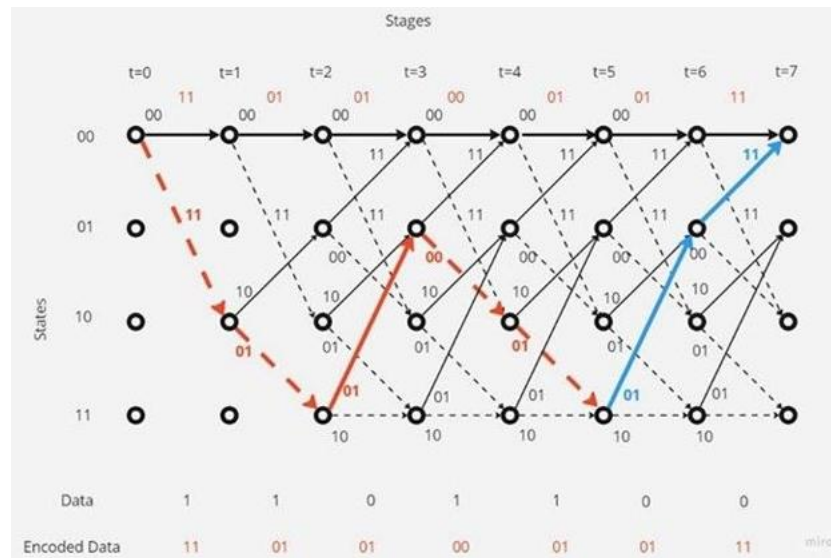
Figure 4: Trellis Diagrams of a Convolutional Code.

Example 9: Consider the convolutional encoder with $k = 1$, $n = 2$ and $K = 3$ shown below. Sketch a trellis diagram to encode the message (1101100)? Consider $g_1 = [111]$ and $g_2 = [101]$.

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0

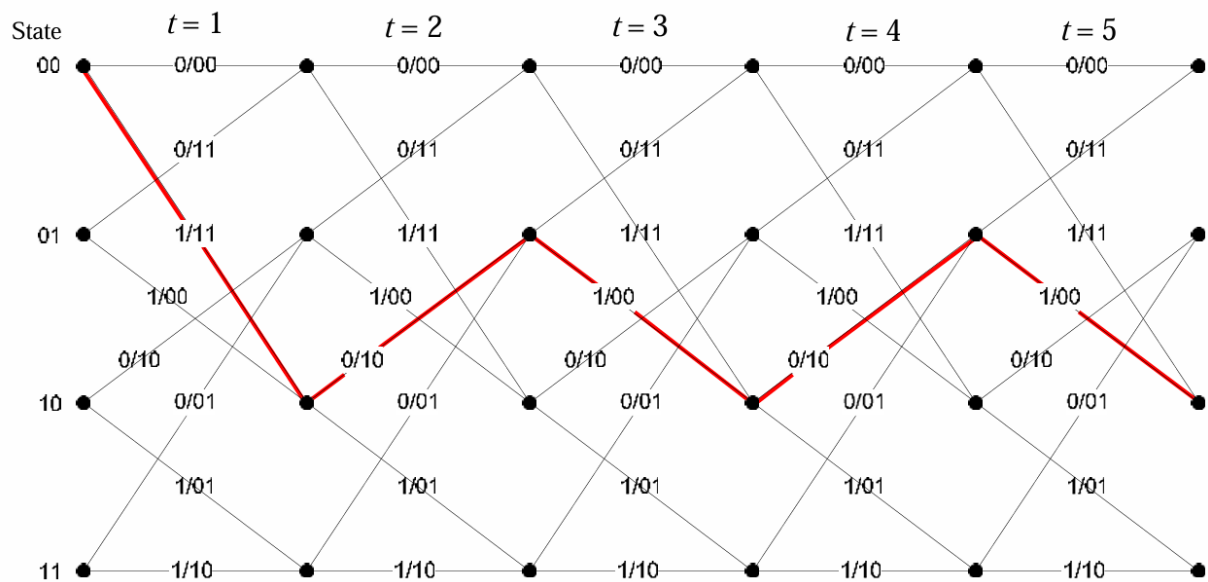


Solution:



Example 10: Consider the convolutional encoder with $k = 1$, $n = 2$, and $K = 3$ shown below. Sketch a trellis diagram to encode the message (10101)? Consider $g_1 = [111]$ and $g_2 = [101]$.

Solution:



The codeword is: 11 10 00 10 00.

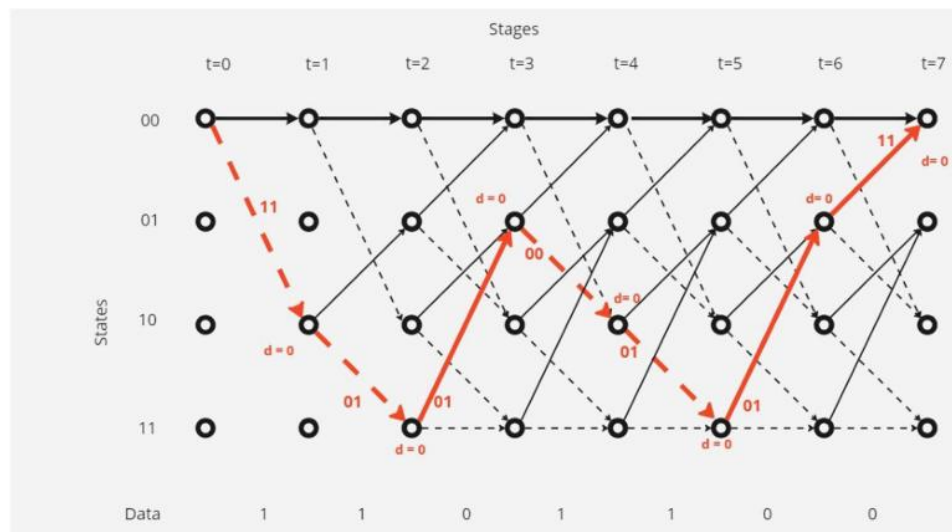
6. Viterbi's Algorithm

As stated previously, it would be impractical to compare every possible codeword with the received word until the codeword with the minimum Hamming (or Euclidean) distance is found.

One maximum-likelihood decoding algorithm, which significantly reduces the number of codewords compared with the received word, is Viterbi's algorithm, introduced by Andrew Viterbi in 1967. Viterbi's algorithm is applied to the code's trellis diagram. It is a recursive algorithm that calculates the accumulated distance between the received word and paths in the trellis at each state. When two paths converge into a state, the least likely path is discarded. Finally, the path of length n with the smallest accumulated distance is the codeword that maximizes $P(r|c)$.

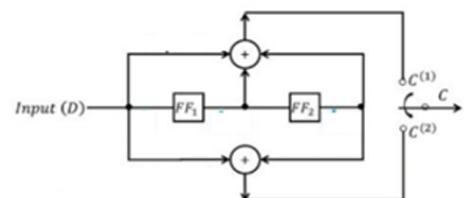
Example 11: Consider the convolutional encoder with $k = 1$, $n = 2$, and $K = 3$. Sketch a trellis diagram to decode the codeword (11010100010111) using hard-Decision Viterbi Decoding? Consider $g_1 = [111]$ and $g_2 = [101]$.

Solution:



Example 12: Consider the convolutional encoder with $k = 1$, $n = 2$, and $K = 3$, the codeword (11010100010111). During the transmission, the 11th bit is flipped. Prove by using hard-Decision Viterbi Decoding, whether the receiver can or not correct the one-bit error? Consider $g_1 = [111]$ and $g_2 = [101]$.

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0



Solution:

